

LAB 1

TITLE: Design of ER Models for Library System and Online Store

OBJECTIVE: To design Entity Relationship (ER) models for the given systems:

1. Library System

2. Online Store

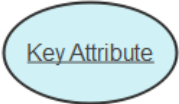
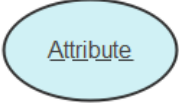
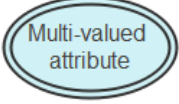
THEORY:

An Entity Relationship (ER) Model is a conceptual representation of the data requirements and structure of a system. It is widely used in database design to visually describe the entities involved in a system, their attributes, and the relationships between them.

Key components of an ER Model:

1. **Entities:** Objects or concepts about which data is stored. Entities can be strong (having a primary key) or weak (dependent on another entity).
2. **Attributes:** Properties or characteristics of an entity. Attributes can be simple, composite, derived, or multivalued.
3. **Relationships:** Associations between entities. They can be one-to-one, one-to-many, or many-to-many.
4. **Cardinality:** Defines the numerical mapping between entities in a relationship.
5. **Primary Keys:** Unique identifiers for each entity instance.

Common symbols used in ER diagrams:

	Entity		Attribute
	Weak Entity		Key Attribute
	Associative Entity		Key Attribute
	Relationship		Key Attribute
	Identifying Relationship		Derived Attribute
	Mandatory Relationship		Optional Relationship
	Partial Participation		Total Participation

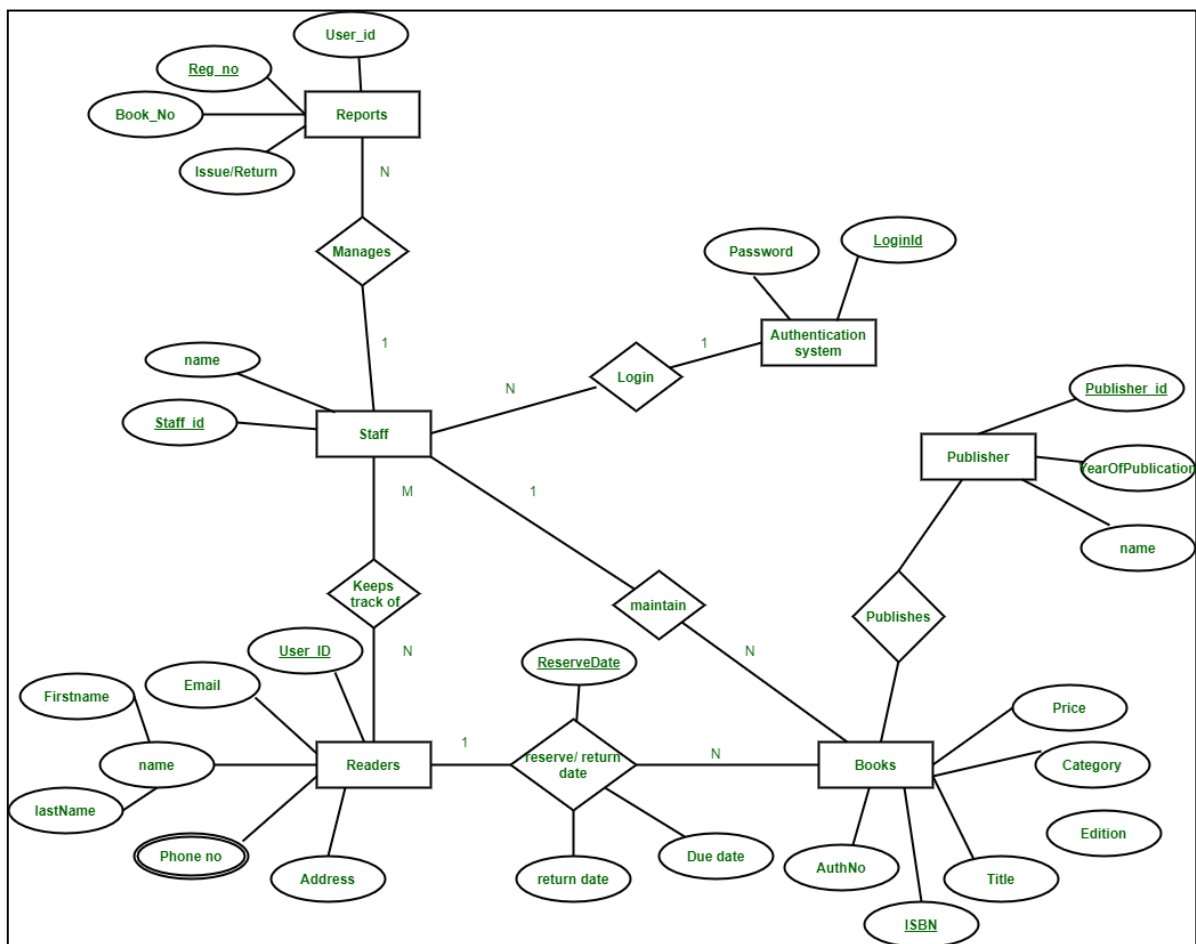


Fig: ER diagram of Library System

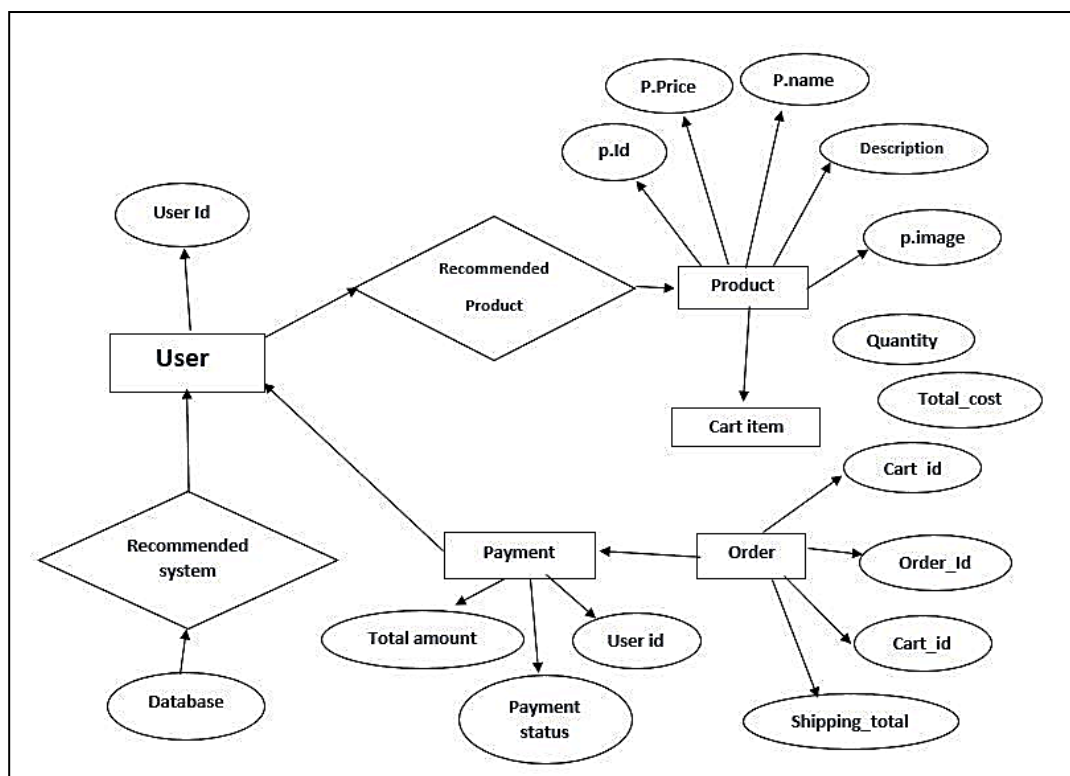


Fig: ER Diagram of Online Store

S. N	Title	Submit Date	Sign
3.	To study different data types in MySQL.	2082/07/15	
4.	To implement the ALTER command in MySQL.	2082/07/15	
5.	To implement the Drop Command in MySQL.	2082/07/15	
6	To query the relational database in order to retrieve the required information by using the join operations.	2082/07/15	
7	To Create database schema and run SQL queries to retrieve the information as instructed.	2082/07/15	
8	To Create Employee Database Schema and Retrieve Information using SQL Aggregate Queries	2082/07/15	
9	TO create database schema and run the different UPDATE, DELETE and ALTER queries using MySQL.	2082/07/15	
10	Creating views in mysql.	2082/07/15	

Experiment No: 3

TITLE: TO STUDY DIFFERENT DATA TYPES IN MYSQL

1. Objectives

- 1.1 To understand the various data types supported by MySQL.
- 1.2 To learn how to use appropriate data types for storing different types of data.
- 1.3 To implement a table using multiple data types and observe their behavior.

2. THEORY

Data types are crucial in MySQL as they determine the kind of values a column can hold, which affects how MySQL stores and processes the data. Choosing the right data type helps optimize storage space, ensures data integrity, and improves performance. MySQL categorizes data types into three main groups: **numeric**, **string**, and **date and time**.

1. Numeric Data Types

Numeric data types are used to store numerical values, including integers, floating-point numbers, and fixed-point numbers.

- **Integer Types**
 - TINYINT: Used for very small integers. It has a signed range of -128 to 127 and an unsigned range of 0 to 255.
 - SMALLINT: A small integer. It has a signed range of -32,768 to 32,767 and an unsigned range of 0 to 65,535.
 - MEDIUMINT: A medium-sized integer. It has a signed range of -8,388,608 to 8,388,607 and an unsigned range of 0 to 16,777,215.
 - INT (or INTEGER): A standard integer. It has a signed range of -2,147,483,648 to 2,147,483,647 and an unsigned range of 0 to 4,294,967,295.
 - BIGINT: Used for very large integers. It has a signed range of -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 and an unsigned range of 0 to 18,446,744,073,709,551,615.
- **Fixed-Point Types**
 - DECIMAL (or NUMERIC): Used for storing precise numerical values where the exact decimal representation is needed, such as for financial data. It's declared with a precision and a scale, for example, DECIMAL(5, 2) can store a number with up to 5 digits in total, with 2 digits after the decimal point.
- **Floating-Point Types**
 - FLOAT: A single-precision floating-point number.
 - DOUBLE: A double-precision floating-point number. FLOAT and DOUBLE are used for storing approximate numeric values.

2. String Data Types

String data types are used to store text, binary data, and other character-based information.

- **Character Types**
 - CHAR: A fixed-length string. If you declare CHAR(10), it will always store 10 characters, padding with spaces if the input is shorter.
 - VARCHAR: A variable-length string. If you declare VARCHAR(255), it will store up to 255 characters, using only the space needed for the actual data.
- **Binary String Types**
 - BINARY: Similar to CHAR, but stores binary byte strings. It's fixed-length.
 - VARBINARY: Similar to VARCHAR, but stores binary byte strings. It's variable-length.
- **Large Text Types**
 - TINYTEXT: A very small text column with a maximum length of 255 characters.
 - TEXT: A small text column with a maximum length of 65,535 characters.
 - MEDIUMTEXT: A medium-sized text column with a maximum length of 16,777,215 characters.
 - LONGTEXT: A large text column with a maximum length of 4,294,967,295 characters.
- **Other String Types**
 - ENUM: An enumeration, which allows you to define a list of possible string values. For example, ENUM('male', 'female', 'other'). This is great for enforcing data integrity.
 - SET: A set, which allows you to store multiple string values from a predefined list. For example, SET('A', 'B', 'C') could store 'A', 'B', 'C', or any combination.
 -

3. Date and Time Data Types

Date and time data types are used to store temporal values.

- DATE: Stores a date in 'YYYY-MM-DD' format.
- TIME: Stores a time in 'HH:MM:SS' format.
- DATETIME: Stores both a date and time in 'YYYY-MM-DD HH:MM:SS' format. This type is not timezone-aware.
- TIMESTAMP: Stores a date and time in 'YYYY-MM-DD HH:MM:SS' format. This type is **timezone-aware**, meaning the value is stored in Coordinated Universal Time (UTC) and converted to the session's timezone upon retrieval. It's often used for tracking creation or modification times.
- YEAR: Stores a year in either 2-digit or 4-digit format.

3. DEMONSTRATION

Given below the table with its datatypes used.

Department Table :

- INT → for unique department ID.
- VARCHAR(100) → for department name and location (variable length strings).
- PRIMARY KEY → ensures each department has a unique ID.

```
MariaDB [maheshudas]> desc department
-> ;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| dept_id    | int(11)       | NO   | PRI | NULL    |       |
| dept_name  | varchar(100)  | NO   |     | NULL    |       |
| location   | varchar(100)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.057 sec)
```

Student Table:

- INT → for student ID and department ID.
- BIGINT → for registration number and contact number (large numeric values).
- VARCHAR(100) → for storing full name.
- ENUM → for fixed gender options (Male, Female, Other).
- DATE → for storing date of birth.
- TEXT → for longer addresses.
- DECIMAL(3,2) → for GPA with precision (e.g., 3.75).
- FOREIGN KEY → links student to department.

```
MariaDB [maheshudas]> desc student;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stud_id    | int(11)       | NO   | PRI | NULL    |       |
| reg_no     | bigint(20)    | YES  | UNI | NULL    |       |
| full_name  | varchar(100)  | NO   |     | NULL    |       |
| gender     | enum('Male', 'Female', 'Other') | YES  |     | NULL    |       |
| dob        | date          | YES  |     | NULL    |       |
| contact    | bigint(20)    | YES  |     | NULL    |       |
| address    | text          | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
| gpa        | decimal(3,2)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

Course Table:

- INT → for course ID and department ID.
- VARCHAR(100) → for course name.
- TINYINT → for credit hours (small integer values).
- FOREIGN KEY → links course to department.

```
MariaDB [maheshudas]> describe course;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| course_id  | int(11)       | NO   | PRI | NULL    |       |
| course_name | varchar(100)  | NO   |     | NULL    |       |
| credit_hours | tinyint(4)    | YES  |     | NULL    |       |
| dept_id    | int(11)       | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

4. RESULT AND DISCUSSION

The tables were successfully created using various MySQL data types such as INT, BIGINT, VARCHAR, TEXT, DECIMAL, ENUM, DATE, DATETIME, and CHAR. The student table demonstrated realistic fields such as registration number (BIGINT), GPA (DECIMAL), gender (ENUM), and address (TEXT). This implementation showed how different data types can be used together to create a practical and well-structured relational database design. Data types for optimized storage and accurate representation of data.

5. CONCLUSION

This experiment provided practical exposure to MySQL data types, emphasizing their importance in database design. Proper selection of numeric, string, and date/time types ensures efficiency, integrity, and accurate data manipulation. Through this exercise, students learned how to implement and work with multiple data types in real-world scenarios.

Experiment No: 4

TITLE: TO IMPLEMENT THE ALTER COMMAND IN MYSQL.

1. OBJECTIVES

- 1.1 To study the purpose and usage of the ALTER command in MySQL.
- 1.2 To learn how to modify the schema of an existing table using ALTER.
- 1.3 To implement actions such as adding, dropping, modifying columns, changing table names.

2. THEORY

In MySQL, the ALTER command is used to modify the structure of an existing table without dropping or recreating it. It provides flexibility to update the schema as database requirements evolve. Using ALTER, one can add new columns, delete unnecessary ones, rename existing columns, or modify their data type and constraints. It can also be used to rename a table entirely, add or remove primary keys, and perform multiple schema changes in a single statement.

The ALTER command is especially useful in real-world database systems where requirements may change frequently. For instance, a company may need to add a new contact column for employees, change the length of a string column to store more data, or rename tables for clarity. Instead of recreating the table, the ALTER command allows changes while preserving existing data.

The ALTER commands are given bellow:

1. Adding a Column

This command is used to add a new column to an existing table. You must specify the name of the new column and its data type.

- Syntax: ALTER TABLE table_name ADD COLUMN column_name datatype;
- Explanation: ALTER TABLE initiates the modification. ADD COLUMN is the action, followed by the new column's name and its datatype.
- Example: ALTER TABLE customers ADD COLUMN email VARCHAR(255);
 - This adds an email column to the customers table. The data type VARCHAR(255) allows it to store variable-length strings up to 255 characters.

2. Removing a Column

This command is used to remove an unwanted column from a table. This action is permanent and will delete all data stored in that column.

- Syntax: ALTER TABLE table_name DROP COLUMN column_name;
- Explanation: DROP COLUMN is the command to delete a column. You simply provide the name of the column you want to remove.

- Example: ALTER TABLE employees DROP COLUMN phone_number;
 - This command removes the phone_number column from the employees table.

3. Modifying a Column

This command is used to change the properties of an existing column, such as its data type or size.

- Syntax: ALTER TABLE table_name MODIFY COLUMN column_name new_datatype;
- Explanation: MODIFY COLUMN tells MySQL to alter the column's definition. You specify the column_name and its new_datatype.
- Example: ALTER TABLE products MODIFY COLUMN price DECIMAL(10, 2);
 - This changes the price column's data type to DECIMAL(10, 2), which is better for storing precise monetary values.

4. Changing a Column's Name and Properties

This is a more powerful command used to rename a column and, optionally, change its properties at the same time.

- Syntax: ALTER TABLE table_name CHANGE COLUMN old_column_name new_column_name new_datatype;
- Explanation: CHANGE COLUMN requires both the old_column_name and the new_column_name, followed by the new datatype.
- Example: ALTER TABLE users CHANGE COLUMN user_name username VARCHAR(50);
 - This command renames the user_name column to username and sets its new data type to VARCHAR(50).

5. Changing a Table's Name

This command is used to rename an entire table.

- Syntax: ALTER TABLE old_table_name RENAME TO new_table_name;
- Explanation: RENAME TO is the clause that handles the table name change. You simply provide the old_table_name and the new_table_name.
- Example: ALTER TABLE customer_info RENAME TO clients;
 - This changes the table name from customer_info to clients.

3. DEMONSTRATIONS

The base table:

```
MariaDB [maheshudas]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
# rows in set (0.068 sec)
```

1. Add a Column

```
MariaDB [maheshudas]> alter table employees add column phone varchar(15);
Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| phone      | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

2. Remove a Column

```
MariaDB [maheshudas]> alter table employees drop column phone;
Query OK, 0 rows affected (0.058 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

3. Modify a Column (Change Data Type or Constraints)

```
MariaDB [maheshudas]> alter table employees modify column phone varchar(20);
Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [ashishsaud]> desc employees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id     | int(11)       | YES  |     | NULL    |       |
| emp_name   | varchar(100)  | YES  |     | NULL    |       |
| department | varchar(50)   | YES  |     | NULL    |       |
| salary     | decimal(10,2) | YES  |     | NULL    |       |
| phone      | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
```

4. Change Column Name and Properties

```
MariaDB [maheshudas]>alter table employees change phone contact_number varchar(20);

Query OK, 0 rows affected (0.040 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc employees;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | YES | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

5. Change Table Name

```
MariaDB [maheshudas]>
MariaDB [maheshudas]> alter table employees rename to staff;
Query OK, 0 rows affected (0.046 sec)
```

6. Add a Primary Key

```
MariaDB [maheshudas]> alter table staff add primary key (emp_id);
Query OK, 0 rows affected (0.063 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]> desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | PRI | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

7. Drop Primary Key

```
MariaDB [maheshudas]>alter table staff drop primary key;
Query OK, 0 rows affected (0.079 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| contact_number | varchar(20) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

8. Perform Multiple Actions in a Single Statement

```
MariaDB [maheshudas]>alter table staff add column age int, drop column contact_number;
Query OK, 0 rows affected (0.043 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshudas]>desc staff;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| emp_id | int(11) | NO | | NULL | |
| emp_name | varchar(100) | YES | | NULL | |
| department | varchar(50) | YES | | NULL | |
| salary | decimal(10,2) | YES | | NULL | |
| age | int(11) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.057 sec)
```

4. RESULT AND DISCUSSION

The experiment demonstrated how the ALTER command can be used to update the schema of an existing table. We successfully added new columns, dropped unnecessary ones, modified data types, changed column names, and even renamed the table. The use of constraints such as primary keys was also shown. This illustrates that ALTER is a versatile command for schema evolution in MySQL.

5. CONCLUSION

The ALTER command is one of the most powerful tools in SQL for managing changes in table structures. It allows database designers to adapt to changing requirements without losing data. This experiment highlighted various operations like adding/removing columns, modifying properties, and renaming tables. Understanding and applying ALTER properly ensures efficient and flexible database management.

Experiment No: 5

TITLE: TO IMPLEMENT THE DROP COMMAND IN MYSQL.

1. OBJECTIVES

- 1.1 To understand the use of the DROP command in MySQL.
- 1.2 To learn how to remove database objects like tables, databases, and constraints.
- 1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

The DROP command in MySQL is a Data Definition Language (DDL) statement used to **delete an entire database, table, or view**. It's a permanent and irreversible action that removes the object and all its associated data and structure from the database.

A. Drop database

This command is used to **delete an entire database**, including all its tables, views, and data. This action cannot be undone, so use it with extreme caution.

- **Syntax:** DROP DATABASE [IF EXISTS] database_name;
- **Explanation:** The IF EXISTS clause is optional but highly recommended. It prevents an error from occurring if you try to drop a database that doesn't exist.
- **Example:** DROP DATABASE IF EXISTS old_project;
 - This command will delete the database named old_project if it exists.

B. Drop table

This command is used to **delete a table** and all the data within it. This action is also permanent and should be used carefully.

- **Syntax:** DROP TABLE [IF EXISTS] table_name;
- **Explanation:** Similar to DROP DATABASE, the IF EXISTS clause prevents an error if the specified table doesn't exist.
- **Example:** DROP TABLE IF EXISTS temporary_users;
 - This command will delete the temporary_users table and all its rows.

3. DEMONSTRATIONS

```
MariaDB [(none)]> create database college;
Query OK, 1 row affected (0.002 sec)

MariaDB [(none)]>
MariaDB [(none)]> use collage;
ERROR 1049 (42000): Unknown database 'collage'
MariaDB [(none)]> use college;
Database changed
```

Database creation :

Table creation:

```
MariaDB [college]> desc student ; desc course;
```

Field	Type	Null	Key	Default	Extra
stud_id	int(11)	NO	PRI	NULL	
name	varchar(100)	YES		NULL	
age	int(11)	YES		NULL	
major	varchar(50)	YES		NULL	

4 rows in set (0.022 sec)

Field	Type	Null	Key	Default	Extra
course_id	int(11)	NO	PRI	NULL	
course_name	varchar(100)	YES		NULL	
credits	int(11)	YES		NULL	

3 rows in set (0.012 sec)

Drop table:

```
MariaDB [college]> drop table course;
Query OK, 0 rows affected (0.057 sec)

MariaDB [college]> drop table student;
Query OK, 0 rows affected (0.013 sec)

MariaDB [college]> desc student ; desc course;
ERROR 1146 (42S02): Table 'college.student' doesn't exist
ERROR 1146 (42S02): Table 'college.course' doesn't exist
```

Drop database:

```
MariaDB [(none)]>
MariaDB [(none)]> drop database college;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> show databases;
```

Database
3daysworkshop
ashishhlab2
ashishlab2
ashishsaud
e_store
information_schema
lab2student
mysql
performance_schema
phpmyadmin
test

11 rows in set (0.001 sec)

4. CONCLUSION

The DROP command is a powerful DDL statement in MySQL used to delete database objects permanently. While it is simple and effective, it should be used with caution as it leads to irreversible data and schema loss.

Experiment No: 6

TITLE: TO QUERY THE RELATIONAL DATABASE IN ORDER TO RETRIEVE THE REQUIRED INFORMATION BY USING THE JOIN OPERATIONS.

1. OBJECTIVES

- 1.1 To understand the use of the DROP command in MySQL.
- 1.2 To learn how to remove database objects like tables, databases, and constraints.
- 1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

JOIN operations in SQL are used to retrieve data from two or more tables by linking them together. This is a fundamental concept in relational databases, allowing you to create a single result set from fragmented data. The common link is usually a **foreign key** column that references the primary key of another table.

Types of JOINS

A. INNER JOIN

The INNER JOIN returns only the rows where the join condition is true in **both** tables. It's the most common type of join and is used to find the intersection of data between two tables.

- **Syntax:** SELECT columns FROM table1 INNER JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** This query will only show rows where the value in table1.column_name is exactly the same as the value in table2.column_name.
- **Analogy:** Imagine you have a list of students and a list of courses. An INNER JOIN would give you a list of students who are **currently enrolled** in a course.

B. LEFT JOIN

The LEFT JOIN (or LEFT OUTER JOIN) returns **all rows from the left table** and the **matching rows from the right table**. If there is no match in the right table, the columns from that table will be filled with NULL.

- **Syntax:** SELECT columns FROM table1 LEFT JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** The table1 (the "left" table) is the one listed first in the query. The result set will contain every row from table1, whether it has a match in table2 or not.
- **Analogy:** If you want a list of every student in the school (left table) and the name of the course they're taking (right table), a LEFT JOIN would show all students, even those who aren't currently enrolled in any course.

C. RIGHT JOIN

The RIGHT JOIN (or RIGHT OUTER JOIN) is the opposite of LEFT JOIN. It returns **all rows from the right table** and the **matching rows from the left table**. Unmatched rows from the left table will have NULL values.

- **Syntax:** SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column_name = table2.column_name;
- **Explanation:** The table2 (the "right" table) is the one listed after the RIGHT JOIN clause. The result will contain every row from table2, regardless of whether a match exists in table1.
- **Analogy:** Using the student and course example, a RIGHT JOIN would give you a list of every course offered (right table) and the names of the students enrolled in them. Courses with no students would still be on the list.

D. FULL JOIN

The FULL JOIN (or FULL OUTER JOIN) returns **all rows when there is a match in either the left or right table**. It essentially combines the results of both LEFT JOIN and RIGHT JOIN, showing all rows from both tables.

- **Syntax (Simulated in MySQL):** SELECT columns FROM table1 LEFT JOIN table2 ON table1.column = table2.column UNION SELECT columns FROM table1 RIGHT JOIN table2 ON table1.column = table2.column WHERE table1.column IS NULL;
- **Explanation:** MySQL doesn't directly support FULL JOIN with a single keyword. The common method is to use a UNION of a LEFT JOIN and a RIGHT JOIN. The UNION operator combines the results of the two select statements.
- **Analogy:** A FULL JOIN would provide a complete list of every student **and** every course, regardless of whether a student is enrolled in a course or a course has any students.

3. DEMONSTRATIONS

Schema: **STUDENT(sid, name, address, contact)**
 BOOK(bid, title, author)
 BURROW(sid, bid)

```
MariaDB [maheshudas]> desc student; desc book; desc burrow;
```

Field	Type	Null	Key	Default	Extra
sid	int(11)	NO	PRI	NULL	
name	text	YES		NULL	
address	text	YES		NULL	
contact	text	YES		NULL	

4 rows in set (0.056 sec)

Field	Type	Null	Key	Default	Extra
bid	int(11)	NO	PRI	NULL	
title	text	YES		NULL	
author	text	YES		NULL	

3 rows in set (0.026 sec)

Field	Type	Null	Key	Default	Extra
sid	int(11)	YES	MUL	NULL	
bid	int(11)	YES	MUL	NULL	

2 rows in set (0.010 sec)

Queries:

a. All the information from student inner join burrow.

```
MariaDB [maheshudas]> SELECT * FROM student INNER JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209

9 rows in set (0.001 sec)

b. All the information from student left join burrow.

```
MariaDB [maheshudas]> SELECT * FROM student LEFT JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
107	ritu thapa	chitwan	9823456789	NULL	NULL
108	binod rai	illam	9801010101	NULL	NULL
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209
110	suman bhattarai	pokhara	9865432100	NULL	NULL

c. All the information from student right join burrow.

```
MariaDB [maheshudas]>SELECT * FROM student RIGHT JOIN burrow ON student.sid=burrow.sid;
```

sid	name	address	contact	sid	bid
101	ashish saud	pokhara	9868845398	101	201
101	ashish saud	pokhara	9868845398	101	202
102	prashant dahal	kathmandu	9841234567	102	203
103	sunita shrestha	biratnagar	9851122334	103	204
104	prajwal bhandari	birtamod	9801230123	104	205
105	diya kc	butwal	9860123456	105	206
106	manoj gurung	dharan	9815012345	106	207
109	samikshya ghimire	damak	9843210987	109	208
109	samikshya ghimire	damak	9843210987	109	209

```
9 rows in set (0.001 sec)
```

d. Name of the students burrowing at least one book.

```
MariaDB [maheshudas]> SELECT DISTINCT name FROM student INNER JOIN burrow ON student.sid=burrow.sid;
```

name
ashish saud
prashant dahal
sunita shrestha
prajwal bhandari
diya kc
manoj gurung
samikshya ghimire

```
7 rows in set (0.026 sec)
```

e. Title of all the books that are burrowed.

```
MariaDB [maheshudas]> SELECT DISTINCT title FROM book INNER JOIN burrow ON book.bid=burrow.bid;
```

title
the alchemist
siddhartha
the hobbit
a song of ice and fire
the great gatsby
pride and prejudice
moby dick
the lord of the rings
1984

```
9 rows in set (0.026 sec)
```

f. Name of the students burrowing no books.

```
MariaDB [maheshudas]> SELECT name FROM student LEFT JOIN burrow ON student.sid=burrow.sid WHERE bid is NULL;
```

name
ritu thapa
binod rai
suman bhattarai

```
3 rows in set (0.001 sec)
```

g. Title of all the books that are not borrowed yet.

```
MariaDB [maheshudas]> SELECT title FROM book LEFT JOIN  
burrow ON book.bid = burrow.bid WHERE sid is NULL;  
+-----+  
| title |  
+-----+  
| to kill a mockingbird |  
+-----+  
1 row in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The queries executed successfully and produced the required results. The INNER JOIN query displayed students who have borrowed books along with their details. The LEFT JOIN retrieved all students, including those who have not borrowed any books, with NULL values in unmatched cases. Similarly, the RIGHT JOIN retrieved all borrowed book entries along with corresponding student details. Queries with DISTINCT ensured that repeated names or titles were not displayed multiple times. Finally, the queries using WHERE ... IS NULL helped identify students who have not borrowed any books and books that are still unborrowed. This demonstrated the importance of different join operations in extracting specific relationships from a relational database.

6. CONCLUSION

This experiment successfully demonstrated the use of different join operations in MySQL. By using INNER JOIN, LEFT JOIN, and RIGHT JOIN, along with filtering conditions, we were able to retrieve important information such as students borrowing books, students not borrowing books, borrowed books, and books that are still unborrowed. Thus, join operations are essential tools for querying multiple tables in relational databases effectively.

Experiment No: 7

TITLE: TO CREATE THE FOLLOWING DATABASE SCHEMA AND RUN SQL QUERIES TO RETRIEVE THE INFORMATION AS INSTRUCTED.

1. Objectives

- 1.1 To design and create an employee table in MySQL.
- 1.2 To apply SQL queries for data retrieval using aliasing, sorting, and conditions.
- 1.3 To understand filtering, range-based conditions, and computed columns in SQL.

2. THEORY

Structured Query Language (SQL) is used to manage and query relational databases. For this experiment, we design an employee table to store details such as ID, name, post, salary, date of birth, gender, address, and contact. SQL provides powerful features such as **aliases**, which help rename columns or tables for better readability, and **conditional operators** like BETWEEN, NOT BETWEEN, and ORDER BY for filtering and sorting results.

SQL Queries

Here are the SQL queries to retrieve the requested information based on the employee table schema.

- **Name, post, and salary of the employee using column alias and table alias.**
 - **Query:** SELECT e.name AS employee_name, e.post AS job_title, e.salary AS monthly_salary FROM employee AS e;
 - **Explanation:** This query selects the name, post, and salary columns from the employee table. It uses e as a table alias for employee to shorten the query. It also uses column aliases (employee_name, job_title, monthly_salary) to provide more descriptive names for the output columns.
- **List of different posts.**
 - **Query:** SELECT DISTINCT post FROM employee;
 - **Explanation:** The DISTINCT keyword is used to retrieve a unique list of values from the post column, avoiding duplicates.
- **List out name, salary, and tax amount (15% of salary).**
 - **Query:** SELECT name, salary, salary * 0.15 AS tax_amount FROM employee;
 - **Explanation:** This query calculates a new column on the fly by multiplying the salary by 0.15 (15%). The result of this calculation is given an alias, tax_amount, to make the column header more readable.
- **Name of all the employees in ascending order.**
 - **Query:** SELECT name FROM employee ORDER BY name ASC;
 - **Explanation:** The ORDER BY clause sorts the result set by the name column. The ASC keyword specifies an ascending order (A to Z), which is the default, so it can also be omitted.
- **Name of the employee having salary between 40,000 and 50,000.**

- **Query:** SELECT name FROM employee WHERE salary BETWEEN 40000 AND 50000;
- **Explanation:** The WHERE clause filters the rows. The BETWEEN operator selects all rows where the salary value is within the specified inclusive range of 40,000 and 50,000.
- **Name of the employee having salary not between 40,000 and 50,000.**
 - **Query:** SELECT name FROM employee WHERE salary NOT BETWEEN 40000 AND 50000;
 - **Explanation:** The NOT BETWEEN operator is the inverse of BETWEEN. It filters for all rows where the salary value falls outside of the inclusive range of 40,000 and 50,000.

3. DEMONSTRATION

Employee table:

```
MariaDB [maheshlabreport]> desc employee;
```

Field	Type	Null	Key	Default	Extra
eid	int(11)	NO	PRI	NULL	
name	text	YES		NULL	
post	text	YES		NULL	
salary	int(11)	YES		NULL	
dob	text	YES		NULL	
gender	text	YES		NULL	
address	text	YES		NULL	
contact	text	YES		NULL	

8 rows in set (0.025 sec)

Inserted Data:

```
MariaDB [maheshlabreport]> select * from employee;
```

eid	name	post	salary	dob	gender	address	contact
101	rajesh sharma	manager	75000	1985-04-12	male	kathmandu	9841001234
102	sita basnet	developer	45000	1992-08-25	female	pokhara	9801234567
103	narayan karki	accountant	55000	1988-11-03	male	biratnagar	9851122334
104	isha thapa	developer	48000	1995-02-14	female	lalitpur	9865432109
105	sanjay rai	team lead	65000	1989-06-30	male	bhaktapur	9815012345
106	pratima dhakal	hr manager	60000	1987-10-10	female	chitwan	9803456789
107	bikram tamang	developer	42000	1993-01-20	male	dharan	9823456789
108	pujan magar	accountant	52000	1990-09-05	male	birtamod	9845678901
109	aayusha ghimire	developer	40000	1994-03-08	female	kathmandu	9801234560
110	sushil gurung	hr manager	62000	1986-07-22	male	pokhara	9865432101

10 rows in set (0.001 sec)

SQL queries to retrieve the following information.

1. Name, post and salary of the employee using column alias and table alias.

```
MariaDB [maheshlabreport]> select name as employee_name, post as employee_post, salary as monthly_salary from employee as e;
```

employee_name	employee_post	monthly_salary
rajesh sharma	manager	75000
sita basnet	developer	45000
narayan karki	accountant	55000
isha thapa	developer	48000
sanjay rai	team lead	65000
pratima dhakal	hr manager	60000
bikram tamang	developer	42000
pujan magar	accountant	52000
aayusha ghimire	developer	40000
sushil gurung	hr manager	62000

10 rows in set (0.001 sec)

2. List of different posts.

```
MariaDB [maheshlabreport]> select distinct post from employee;
+-----+
| post  |
+-----+
| manager      |
| developer    |
| accountant   |
| team lead    |
| hr manager   |
+-----+
5 rows in set (0.022 sec)
```

3. List out name, salary and tax amount (15% of salary).

```
MariaDB [maheshlabreport]> select name, salary, salary * 0.15 as tax_amount from employee;
+-----+-----+-----+
| name      | salary | tax_amount |
+-----+-----+-----+
| rajesh sharma | 75000 | 11250.00 |
| sita basnet  | 45000 | 6750.00  |
| narayan karki | 55000 | 8250.00  |
| isha thapa   | 48000 | 7200.00  |
| sanjay rai    | 65000 | 9750.00  |
| pratima dhakal | 60000 | 9000.00  |
| bikram tamang | 42000 | 6300.00  |
| puja magar    | 52000 | 7800.00  |
| aayusha ghimire | 40000 | 6000.00  |
| sushil gurung | 62000 | 9300.00  |
+-----+-----+-----+
10 rows in set (0.001 sec)
```

4. Name of all the employees in ascending order.

```
MariaDB [maheshlabreport]> select name from employee order by name asc;
+-----+
| name      |
+-----+
| aayusha ghimire |
| bikram tamang   |
| isha thapa      |
| narayan karki   |
| pratima dhakal  |
| puja magar      |
| rajesh sharma   |
| sanjay rai      |
| sita basnet     |
| sushil gurung   |
+-----+
10 rows in set (0.001 sec)
```

5. Name of the employee having salary between 40,000 and 50,000.

```
MariaDB [maheshlabreport]> select name from employee where salary between 40000 and 50000;
+-----+
| name      |
+-----+
| sita basnet |
| isha thapa  |
| bikram tamang |
| aayusha ghimire |
+-----+
4 rows in set (0.001 sec)
```

6. Name of the employee having salary not between 40,000 and 50,000.

```
MariaDB [maheshlabreport]> select name from employee where salary not between 40000 and 50000;
+-----+
| name |
+-----+
| rajesh sharma |
| narayan karki |
| sanjay rai |
| pratima dhakal |
| pujan magar |
| sushil gurung |
+-----+
6 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The queries were executed on the employee table successfully. Using table and column aliases, employee details such as name, post, and salary were displayed with more descriptive labels. The distinct keyword listed different posts without repetition. The computation $\text{salary} \times 0.15$ correctly displayed the tax amount for each employee. Sorting with order by arranged the employees' names in ascending order. The use of between and not between conditions helped filter employees based on their salary range. This demonstrated the practical use of SQL clauses for efficient data retrieval.

5. CONCLUSION

This experiment successfully demonstrated the creation of an employee database and execution of various SQL queries. We learned to use aliases for readability, distinct values for uniqueness, computed columns for calculations, sorting for ordered results, and conditional queries for filtering based on ranges. Overall, this enhanced understanding of SQL's querying power in handling real-world database operations.

Experiment No: 8

TITLE: TO CREATE EMPLOYEE DATABASE SCHEMA AND RETRIEVE INFORMATION USING SQL AGGREGATE QUERIES

1. OBJECTIVES

- 1.2 To implement aggregate functions like count(), avg(), and sum().
- 1.3 To use subqueries for retrieving maximum and comparison-based data.
- 1.4 To analyze salary distribution among employees using SQL.

2. THEORY

In SQL, **aggregate functions** are used to perform calculations on sets of values and return a single summarizing value. Functions such as COUNT(), AVG(), and SUM() are essential for analyzing data in a database. These functions help determine the number of records, average values, or total amounts. SQL also supports the use of **GROUP BY** to organize records into groups, and **subqueries** to retrieve values for comparison against other queries.

A key companion to aggregate functions is the GROUP BY clause. This clause is used to group rows that have the same values in specified columns into summary rows. For example, by using GROUP BY, we can calculate the average salary for each distinct job post or for each gender, rather than just for the entire table.

Additionally, we will demonstrate the use of a **subquery**. A subquery is a query nested inside another query. The inner query, or subquery, executes first, and its result is used by the outer query. This is a powerful technique that allows for more complex data retrieval, such as finding the name of an employee who has the single highest salary in the entire table.

SQL Queries

Here are the SQL queries to retrieve the requested information using aggregate functions based on the employee table schema.

- **Count the total number of employees in the 'employee' table.**
 - **Query:** SELECT COUNT(*) FROM employee;
 - **Explanation:** The COUNT(*) function counts all rows in the employee table to give the total number of employees.
- **Find the average salary of all employees.**
 - **Query:** SELECT AVG(salary) FROM employee;
 - **Explanation:** The AVG() function calculates the average value of the salary column across all employees.
- **Find the average salary of male and female employees.**
 - **Query:** SELECT gender, AVG(salary) FROM employee GROUP BY gender;

- **Explanation:** This query uses AVG(salary) to calculate the average salary, but the GROUP BY gender clause ensures that the calculation is performed separately for each unique value in the gender column (e.g., 'male' and 'female').
- **Find the total number of male and female employees and their average salary.**
 - **Query:** SELECT gender, COUNT(*) AS total_employees, AVG(salary) AS average_salary FROM employee GROUP BY gender;
 - **Explanation:** This query combines two aggregate functions: COUNT(*) to count the employees in each group and AVG(salary) to find their average salary. The GROUP BY gender clause organizes the data by gender, and column aliases are used to make the output headers more descriptive.
- **Find the name, post, and salary of the employee having the maximum salary (using a subquery).**
 - **Query:** SELECT name, post, salary FROM employee WHERE salary = (SELECT MAX(salary) FROM employee);
 - **Explanation:** This query first uses a subquery (SELECT MAX(salary) FROM employee) to find the single highest salary value in the table. The outer query then uses the WHERE clause to filter the employee table and return the details of the employee whose salary matches that maximum value.
- **Find the total amount the company must pay as salary per month.**
 - **Query:** SELECT SUM(salary) FROM employee;
 - **Explanation:** The SUM() function adds up all the values in the salary column to provide the total amount the company pays in salaries.

3. DEMONSTRATIONS

The base table:

```
MariaDB [maheshlabreport]> create table employee (eid integer primary key, name
text, post text, salary integer, gender text, address text, contact text);
Query OK, 0 rows affected (0.040 sec)

MariaDB [ashishlabreport]> desc employee;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| eid   | int(11) | NO   | PRI | NULL    |       |
| name  | text   | YES  |     | NULL    |       |
| post  | text   | YES  |     | NULL    |       |
| salary | int(11) | YES  |     | NULL    |       |
| gender | text   | YES  |     | NULL    |       |
| address | text   | YES  |     | NULL    |       |
| contact | text   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.025 sec)
```

1. Count the total number of employees in the 'employee' table.

```
MariaDB [maheshlabreport]> select count(*) from employee;
+-----+
| count(*) |
+-----+
|      10 |
+-----+
1 row in set (0.001 sec)
```

2. Find the average salary of all employees.

```
MariaDB [maheshlabreport]> select avg(salary) from employee;
+-----+
| avg(salary) |
+-----+
| 54400.0000 |
+-----+
1 row in set (0.001 sec)
```

3. Find the average salary of male and female employees.

```
MariaDB [maheshlabreport]> select gender, avg(salary) from employee group by gender;
+-----+-----+
| gender | avg(salary) |
+-----+-----+
| female | 48250.0000 |
| male   | 58500.0000 |
+-----+-----+
2 rows in set (0.020 sec)
```

4. Find the total number of male and female employees and their average salary.

```
MariaDB [maheshlabreport]> select gender, count(*), avg(salary) from employee group by gender;
+-----+-----+-----+
| gender | count(*) | avg(salary) |
+-----+-----+-----+
| female |      4 | 48250.0000 |
| male   |      6 | 58500.0000 |
+-----+-----+-----+
2 rows in set (0.025 sec)
```

5. Find the name post and salary of the employee having maximum salary (use subquery)

```
MariaDB [maheshlabreport]> select name, post, salary from employee where salary
= (select max(salary) from employee);
+-----+-----+-----+
| name      | post    | salary |
+-----+-----+-----+
| rajesh sharma | manager | 75000 |
+-----+-----+-----+
1 row in set (0.001 sec)
```

6. Find the total amount the company must pay as salary per month.

```
MariaDB [maheshlabreport]> select sum(salary) from employee;
+-----+
| sum(salary) |
+-----+
|      544000 |
+-----+
1 row in set (0.001 sec)
```

7. How much does the company pay to all the females as salary?

```
MariaDB [maheshlabreport]> select sum(salary) from employee where gender = 'female';
+-----+
| sum(salary) |
+-----+
|      193000 |
+-----+
1 row in set (0.001 sec)
```

8. Find the name of an employee having a salary less than average.

```
MariaDB [maheshlabreport]> select name from employee where salary < (select avg(salary) from employee);
+-----+
| name |
+-----+
| sita basnet |
| isha thapa |
| bikram tamang |
| pujan magar |
| aayusha ghimire |
+-----+
5 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The experiment demonstrated how the ALTER command can be used to update the schema of an existing table. We successfully added new columns, dropped unnecessary ones, modified data types, changed column names, and even renamed the table. The use of constraints such as primary keys was also shown. This illustrates that ALTER is a versatile command for schema evolution in MySQL.

5. CONCLUSION

The ALTER command is one of the most powerful tools in SQL for managing changes in table structures. It allows database designers to adapt to changing requirements without losing data. This experiment highlighted various operations like adding/removing columns, modifying properties, and renaming tables. Understanding and applying ALTER properly ensures efficient and flexible database management.

Experiment No: 9

TITLE: TO CREATE DATABASE SCHEMA AND RUN THE DIFFERENT UPDATE, DELETE AND ALTER QUERIES USING MYSQL.

1. OBJECTIVES

- 1.1 To understand the use of UPDATE, DELETE, and ALTER commands in MySQL.
- 1.2 To modify records and structure of a table using different SQL queries.
- 1.3 To analyze how update and delete operations affect the stored data.

2. THEORY

In MySQL, **Data Manipulation Language (DML)** and **Data Definition Language (DDL)** commands are used to manage both data and schema. The UPDATE command modifies existing records in a table, while the DELETE command removes specific records based on conditions. The ALTER command modifies the structure of an existing table by adding, deleting, or renaming columns and even renaming the table itself.

Data Manipulation Language (DML) Queries

1. Increase salary of all employees by 10%.

- **Query:** UPDATE employee SET salary = salary * 1.1;
- **Explanation:** The UPDATE statement modifies existing records in a table. By not including a WHERE clause, this query affects **all** rows in the employee table, multiplying each employee's salary by 1.1 to increase it by 10%.

2. List out different cities and update the postal code for each.

- **Query (Example for Pokhara):** UPDATE employee SET postal_code = 33600 WHERE city = 'pokhara';
- **Explanation:** This UPDATE query uses a WHERE clause to selectively apply changes. It will only update the postal_code column to 33600 for records where the city column is 'pokhara'. You would repeat this for other cities as needed.

3. Add a * sign after the post for employees with above-average salary.

- **Query:** UPDATE employee SET post = CONCAT(post, '*') WHERE salary > (SELECT AVG(salary) FROM employee);
- **Explanation:** This advanced UPDATE query uses a **subquery** (SELECT AVG(salary) FROM employee) to first calculate the average salary. The WHERE clause then filters for employees whose salary is greater than that average. The CONCAT() function is used to append a * to the end of their post string.

4. Delete records of employees from Pokhara and who are male.

- **Query:** DELETE FROM employee WHERE city = 'pokhara' AND gender = 'm';

- **Explanation:** The DELETE statement removes entire rows from a table. The WHERE clause is critical here, as it ensures that only records that meet **both** conditions (city is 'pokhara' AND gender is 'm') are permanently removed.

Data Definition Language (DDL) Queries

1. Add a new column postal_code after city.

- **Query:** ALTER TABLE employee ADD COLUMN postal_code INT AFTER city;
- **Explanation:** The ALTER TABLE command modifies a table's structure. ADD COLUMN creates a new column named postal_code with a data type of INT. The AFTER city clause specifies the new column's position, placing it immediately after the city column.

2. Change the name of the table to emp.

- **Query:** ALTER TABLE employee RENAME TO emp;
- **Explanation:** This ALTER TABLE command renames the table from employee to emp. All data and existing table properties remain the same.

3. Change the name of the column name to full_name.

- **Query:** ALTER TABLE emp CHANGE name full_name VARCHAR(30) NOT NULL;
- **Explanation:** The CHANGE clause in ALTER TABLE is used to rename a column. You must specify the old name (name), the new name (full_name), and its complete new definition (VARCHAR(30) NOT NULL), even if the data type and constraints are staying the same.

3. DEMONSTRATIONS

Table creation:

```
MariaDB [maheshlabreport]> desc employee;
```

Field	Type	Null	Key	Default	Extra
eid	int(11)	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
post	varchar(50)	YES		NULL	
salary	int(11)	YES		NULL	
gender	varchar(10)	YES		NULL	
city	varchar(50)	YES		NULL	
contact	varchar(15)	YES		NULL	

7 rows in set (0.024 sec)

- **Increase salary of all employee by 10%.**

Before:

```
MariaDB [maheshlabreport]> select eid, name, salary from employee;
```

eid	name	salary
101	rajesh sharma	82500
102	sita basnet	49500
103	narayan karki	60500
104	isha thapa	52800
105	sanjay rai	71500
106	pratima dhakal	66000
107	bikram tamang	46200
108	pujan magar	57200
109	aayusha ghimire	44000
110	sushil gurung	68200

```
10 rows in set (0.001 sec)
```

After:

```
MariaDB [maheshlabreport]> update employee set salary = (salary * 1.1);
Query OK, 10 rows affected (0.006 sec)
Rows matched: 10  Changed: 10  Warnings: 0

MariaDB [maheshlabreport]> select eid, name, salary from employee;
```

eid	name	salary
101	rajesh sharma	90750
102	sita basnet	54450
103	narayan karki	66550
104	isha thapa	58080
105	sanjay rai	78650
106	pratima dhakal	72600
107	bikram tamang	50820
108	pujan magar	62920
109	aayusha ghimire	48400
110	sushil gurung	75020

```
10 rows in set (0.001 sec)
```

- Add a new column 'postal_code' after city.

```
MariaDB [maheshlabreport]> alter table employee add column postal_code int after city;
Query OK, 0 rows affected (0.012 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [maheshlabreport]> desc employee;
```

Field	Type	Null	Key	Default	Extra
eid	int(11)	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
post	varchar(50)	YES		NULL	
salary	int(11)	YES		NULL	
gender	varchar(10)	YES		NULL	
city	varchar(50)	YES		NULL	
postal_code	int(11)	YES		NULL	
contact	varchar(15)	YES		NULL	

```
8 rows in set (0.023 sec)
```

- List out different cities in the table using select query and update the postal code for each city.

```

MariaDB [maheshlabreport]> select city from employee;
+-----+
| city |
+-----+
| kathmandu |
| pokhara |
| biratnagar |
| pokhara |
| bhaktapur |
| kathmandu |
| pokhara |
| birtamod |
| kathmandu |
| pokhara |
+-----+
10 rows in set (0.001 sec)

MariaDB [maheshlabreport]> update employee set postal_code = 44600 where city = 'kathmandu';
Query OK, 3 rows affected (0.005 sec)
Rows matched: 3 Changed: 3 Warnings: 0

MariaDB [maheshlabreport]> update employee set postal_code = 33700 where city = 'pokhara';
Query OK, 4 rows affected (0.002 sec)
Rows matched: 4 Changed: 4 Warnings: 0

MariaDB [maheshlabreport]> update employee set postal_code = 56700 where city = 'biratnagar';
Query OK, 1 row affected (0.002 sec)
Rows matched: 1 Changed: 1 Warnings: 0

MariaDB [maheshlabreport]> update employee set postal_code = 44800 where city = 'bhaktapur';
Query OK, 1 row affected (0.002 sec)
Rows matched: 1 Changed: 1 Warnings: 0

MariaDB [maheshlabreport]> update employee set postal_code = 57100 where city = 'birtamod';
Query OK, 1 row affected (0.005 sec)
Rows matched: 1 Changed: 1 Warnings: 0

```

- Add a * sign after the post of those employees who have salary more than average.

```

MariaDB [maheshlabreport]> update employee set post = concat(post, '*') where salary > (select avg(salary)
from (select * from employee) as temp);
Query OK, 5 rows affected (0.007 sec)
Rows matched: 5 Changed: 5 Warnings: 0

MariaDB [maheshlabreport]> select eid , name, post from employee;
+-----+-----+-----+
| eid | name | post |
+-----+-----+-----+
| 101 | rajesh sharma | manager* |
| 102 | sita basnet | developer |
| 103 | narayan karki | accountant* |
| 104 | isha thapa | developer |
| 105 | sanjay rai | team lead* |
| 106 | pratima dhakal | hr manager* |
| 107 | bikram tamang | developer |
| 108 | pujan magar | accountant |
| 109 | aayusha ghimire | developer |
| 110 | sushil gurung | hr manager* |
+-----+-----+-----+
10 rows in set (0.001 sec)

```

- Delete the records of those employees who are from pokhara and are male.

```

MariaDB [maheshlabreport]> delete from employee where city = 'pokhara' and gender = 'male';
Query OK, 2 rows affected (0.006 sec)

MariaDB [maheshlabreport]> select eid , name, city, gender from employee;
+-----+-----+-----+-----+
| eid | name | city | gender |
+-----+-----+-----+-----+
| 101 | rajesh sharma | kathmandu | male |
| 102 | sita basnet | pokhara | female |
| 103 | narayan karki | biratnagar | male |
| 104 | isha thapa | pokhara | female |
| 105 | sanjay rai | bhaktapur | male |
| 106 | pratima dhakal | kathmandu | female |
| 108 | pujan magar | birtamod | male |
| 109 | aayusha ghimire | kathmandu | female |
+-----+-----+-----+-----+
8 rows in set (0.001 sec)

```

- Change the name of the table to 'emp'.

```
MariaDB [maheshlabreport]> alter table employee rename to emp;
Query OK, 0 rows affected (0.014 sec)

MariaDB [maheshlabreport]> show tables;
+-----+
| Tables_in_ashishlabreport |
+-----+
| emp                        |
+-----+
1 row in set (0.001 sec)
```

- Change the name of the column 'name' to 'full_name'.

```
MariaDB [maheshlabreport]> alter table emp change name full_name varchar(50)
Query OK, 0 rows affected (0.012 sec)
Records: 0 Duplicates: 0 Warnings: 0

MariaDB [maheshlabreport]> desc emp;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| eid        | int(11)       | NO   | PRI | NULL    |       |
| full_name  | varchar(50)   | YES  |     | NULL    |       |
| post       | varchar(50)   | YES  |     | NULL    |       |
| salary     | int(11)       | YES  |     | NULL    |       |
| gender     | varchar(10)   | YES  |     | NULL    |       |
| city       | varchar(50)   | YES  |     | NULL    |       |
| postal_code | int(11)       | YES  |     | NULL    |       |
| contact    | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.021 sec)
```

4. RESULT AND DISCUSSION

The experiment was carried out successfully. The UPDATE command increased the salary of all employees by 10%, and specific postal codes were assigned based on the city. A subquery was used to mark employees with above-average salary by appending an asterisk to their post. The DELETE command removed male employees from Pokhara. Using the ALTER command, a new column was added, the table name was changed to emp, and the column name was renamed to full_name. This experiment demonstrated both data-level and schema-level modifications effectively.

5. CONCLUSION

This experiment provided hands-on experience with the UPDATE, DELETE, and ALTER commands in MySQL. We learned how to increase salaries, assign postal codes, mark employees with high salaries, and remove specific records. Additionally, structural changes were applied by renaming the table and columns. These commands are powerful for maintaining and evolving database systems in real-world applications.

Experiment No: 10

TITLE: TO QUERY THE RELATIONAL DATABASE IN ORDER TO RETRIEVE THE REQUIRED INFORMATION BY USING THE JOIN OPERATIONS.

1. OBJECTIVES

- 1.1 To understand the use of the DROP command in MySQL.
- 1.2 To learn how to remove database objects like tables, databases, and constraints.
- 1.3 To analyze the effect of DROP on the schema and stored data.

2. THEORY

A **view** in MySQL is a **virtual table** based on the result of an SQL query. Unlike physical tables, views do not store data themselves; they display data stored in underlying tables. Views are especially useful for simplifying complex queries, enhancing data security by restricting access to sensitive columns, and providing customized representations of data.

Creating a View

- **Syntax:**
 - CREATE VIEW view_name AS
 - SELECT column1, column2, ...
 - FROM table_name
 - WHERE condition;
- **Explanation:**
 - CREATE VIEW view_name starts the command and gives the view a name.
 - AS is a required keyword that separates the view name from the query.
 - SELECT ... is the query that defines what data the view will contain.
- **Key Characteristics:**
 - Defined using the create view statement.
 - Acts like a table but does not store data physically.
 - Can be used in select queries as a normal table.
 - Supports abstraction by hiding complexity of queries.

Advantages:

- Simplifies query writing by encapsulating complex queries.
- Enhances security by allowing restricted access to specific columns.

- Provides logical independence between user queries and underlying schema.

Disadvantages:

- Performance overhead for very complex views.
- Cannot always perform insert, update, or delete on views depending on their definition.

3. DEMONSTRATIONS

Schema: employee(eid, name, post, salary, gender, city))

```
MariaDB [maheshlabreport]> create table employee (eid int primary key, name varchar(100),
post varchar(50), salary decimal(10,2), gender varchar(10), city varchar(50));
Query OK, 0 rows affected (0.015 sec)

MariaDB [maheshlabreport]> desc employee;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| eid   | int(11)       | NO   | PRI | NULL    |       |
| name  | varchar(100)  | YES  |     | NULL    |       |
| post  | varchar(50)   | YES  |     | NULL    |       |
| salary | decimal(10,2) | YES  |     | NULL    |       |
| gender | varchar(10)   | YES  |     | NULL    |       |
| city  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.021 sec)
```

Table data:

```
MariaDB [maheshlabreport]> select * from employee;
+-----+-----+-----+-----+-----+-----+
| eid | name       | post      | salary | gender | city      |
+-----+-----+-----+-----+-----+-----+
| 1   | ram sharma | manager   | 60000.00 | male   | kathmandu |
| 2   | sita rai   | developer | 45000.00 | female | lalitpur   |
| 3   | hari karki | developer | 42000.00 | male   | pokhara    |
| 4   | sophia gurun | analyst  | 38000.00 | female | pokhara    |
| 5   | suman khadka | hr        | 50000.00 | male   | butwal     |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)
```

Create view #1 (simple “basic details” view):

```
MariaDB [maheshlabreport]> create view employee_basic as select eid, name, post from employee;
Query OK, 0 rows affected (0.008 sec)

MariaDB [ashishlabreport]> select * from employee_basic;
+-----+-----+-----+
| eid | name       | post      |
+-----+-----+-----+
| 1   | ram sharma | manager   |
| 2   | sita rai   | developer |
| 3   | hari karki | developer |
| 4   | sophia gurun | analyst  |
| 5   | suman khadka | hr        |
+-----+-----+-----+
5 rows in set (0.002 sec)
```

Create view #2 (high salary employees) :

```
MariaDB [maheshlabreport]> create view high_salary as select name, post, salary from employee where salary > 45000;
Query OK, 0 rows affected (0.009 sec)

MariaDB [maheshlabreport]> select * from high_salary;
+-----+-----+-----+
| name      | post   | salary |
+-----+-----+-----+
| ram sharma | manager | 60000.00 |
| suman khadka | hr     | 50000.00 |
+-----+-----+-----+
2 rows in set (0.002 sec)
```

Create view #3 (female employees):

```
MariaDB [maheshlabreport]> create view female_employees as select name,
city, salary from employee where gender='female';
Query OK, 0 rows affected (0.008 sec)

MariaDB [maheshlabreport]> select * from female_employees;
+-----+-----+-----+
| name      | city    | salary |
+-----+-----+-----+
| sita rai   | lalitpur | 45000.00 |
| sophia gurung | pokhara | 38000.00 |
+-----+-----+-----+
2 rows in set (0.002 sec)
```

list all views in the db:

```
MariaDB [maheshlabreport]> show full tables where table_type='VIEW';
+-----+-----+
| Tables_in_ashishlabreport | Table_type |
+-----+-----+
| employee_basic            | VIEW       |
| female_employees          | VIEW       |
| high_salary                | VIEW       |
+-----+-----+
3 rows in set (0.001 sec)
```

4. RESULT AND DISCUSSION

The experiment was successfully performed by creating three views in the employee schema. The first view, `employee_basic`, displayed only the essential details of employees such as ID, name, and post. The second view, `high_salary`, showed the list of employees earning more than 45,000. The third view, `female_employees`, listed only the female employees with their city and salary. These views simplified queries, restricted access to certain information, and demonstrated the usefulness of views in real-world database applications.

6. CONCLUSION

This experiment demonstrated how to create and use views in MySQL. Views provide a way to simplify queries, enhance security by limiting column access, and offer an abstraction layer between users and the actual database schema. By creating and querying views, the importance of views in database management was clearly understood.